

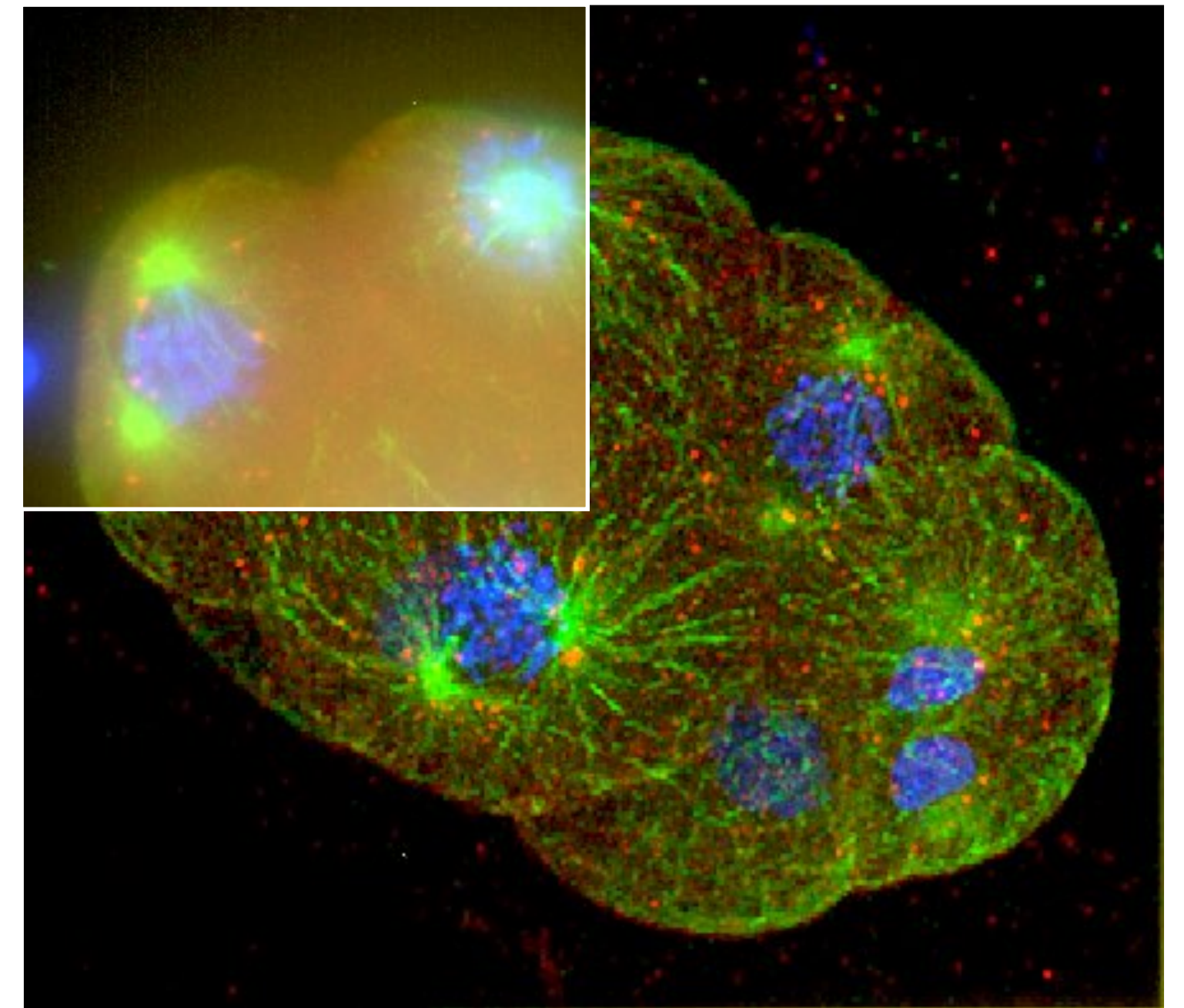
DeconvolutionLab2

Daniel Sage

Biomedical Imaging Group

EPFL Center for Imaging

Ecole Polytechnique Fédérale de Lausanne





PHD

EPFL

PROJECT

EXPERTISE

TEACHING

CODE

daniel.sage@epfl.ch

Image Processing

25+ years at BIG Prof. M. Unser

100+ collaborations with life-scientists ...

Analysis · Tracking · Directionality · CNN ·
Deconvolution · Superes · SMLM

Bioimage Informatics · Doctoral school
Continuing Training

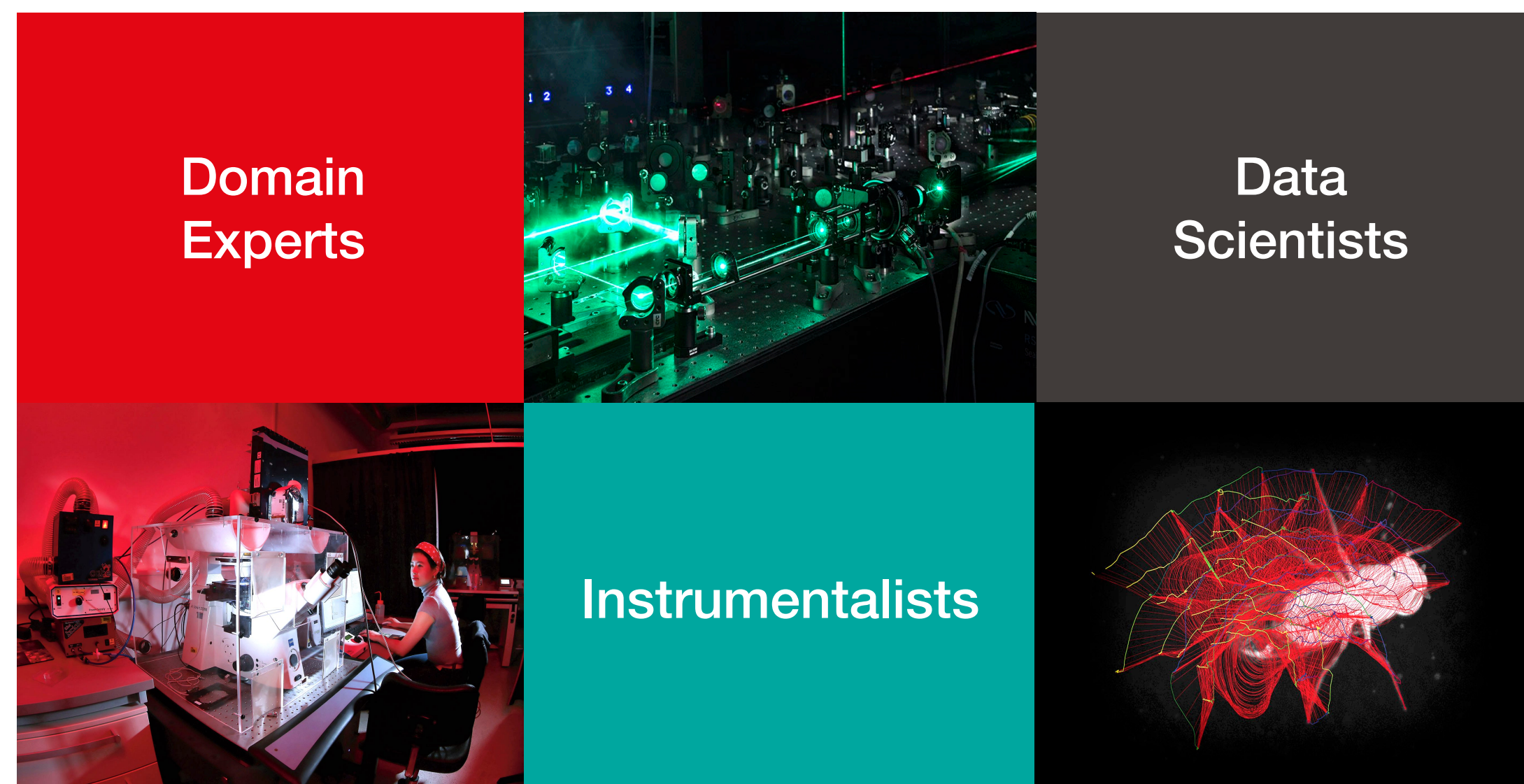
OrientationJ · DL2 · deeplImageJ ·
SAMJ, ...

Daniel Sage

comme une image

EPFL Center for Imaging

Fostering excellence
in imaging across **domains** and across **scales**



Needs are abundant and similar ➤ Support with image data
Required imaging skills ➤ Training PhD Industry, Master

Deconvolution

“In mathematics, deconvolution is an **algorithm-based process** used to **enhance signals** from recorded data. Where the recorded data can be **modeled by a convolution**”

Wikipedia

“Deconvolution is a **computational method** to **improve digital image quality** by using **knowledge of the way the microscope forms images**”

David Agard



“Deconvolution is a **computationally intensive image processing technique** used to **improve the contrast and sharpness of images captured using a microscope**”

Olympus

Goals

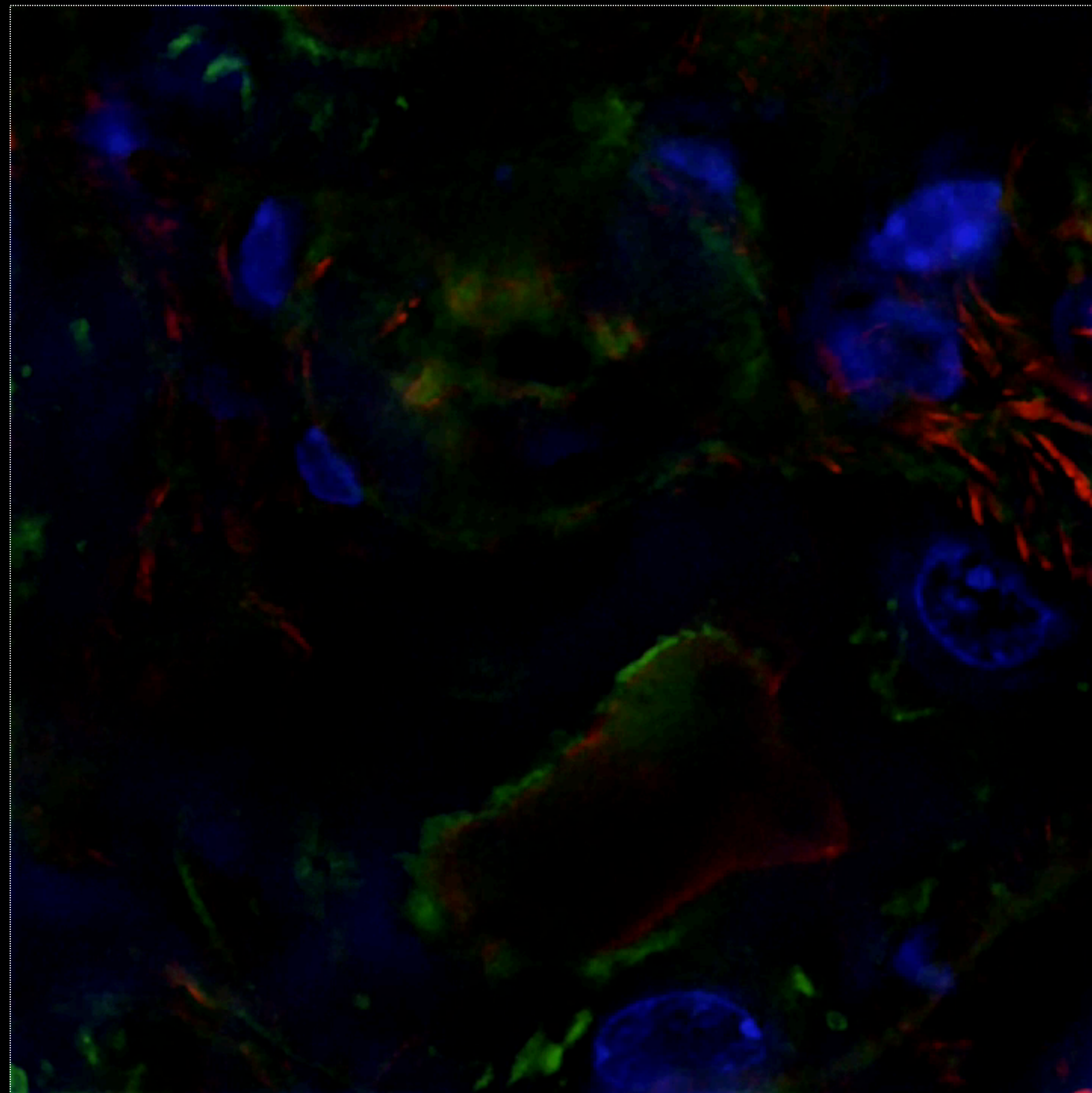
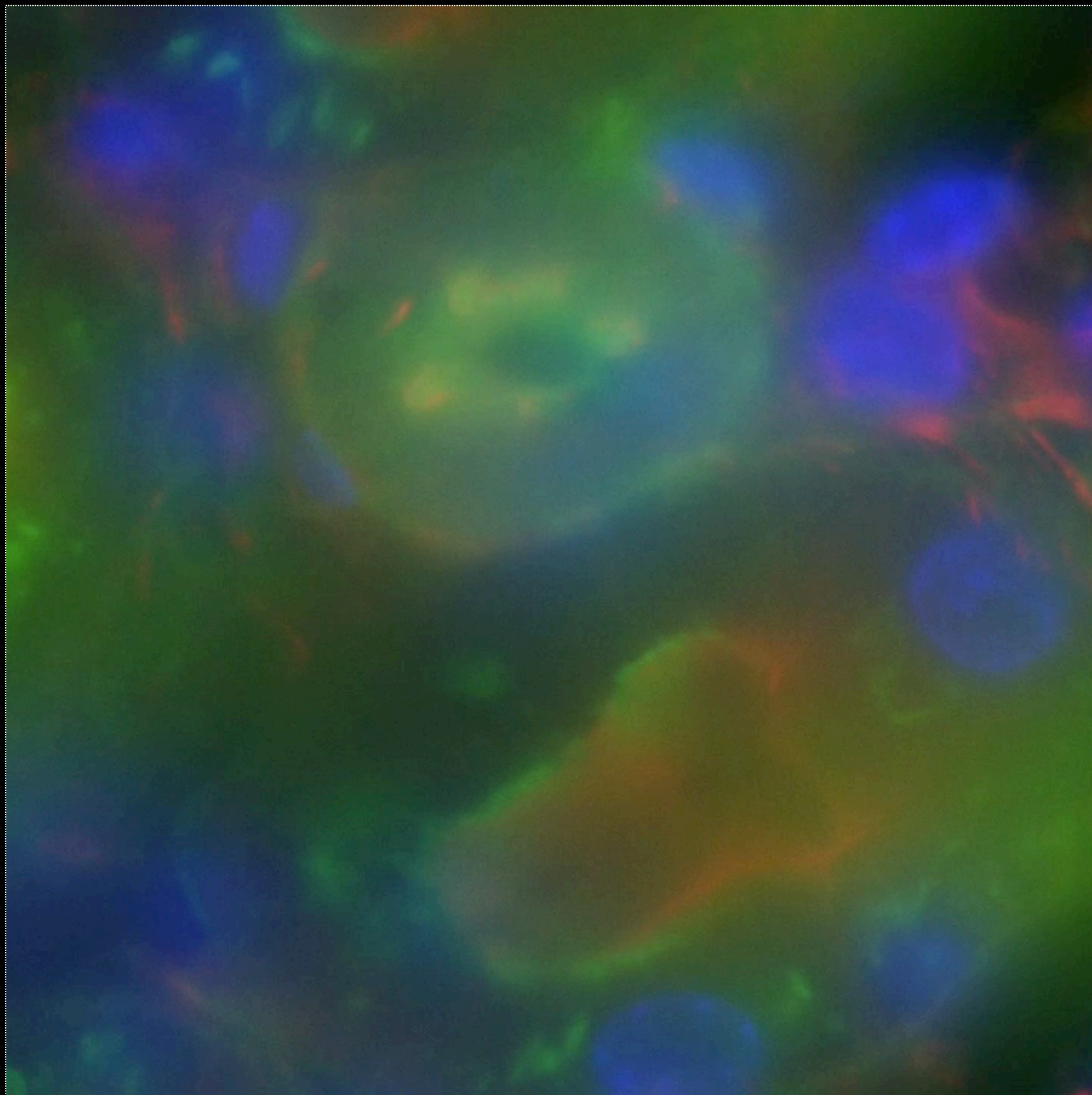
Improve quality
Enhance contrast
Improve resolution?

Image formation model

Image formation models
Using knowledge
Convolution model

Computational methods

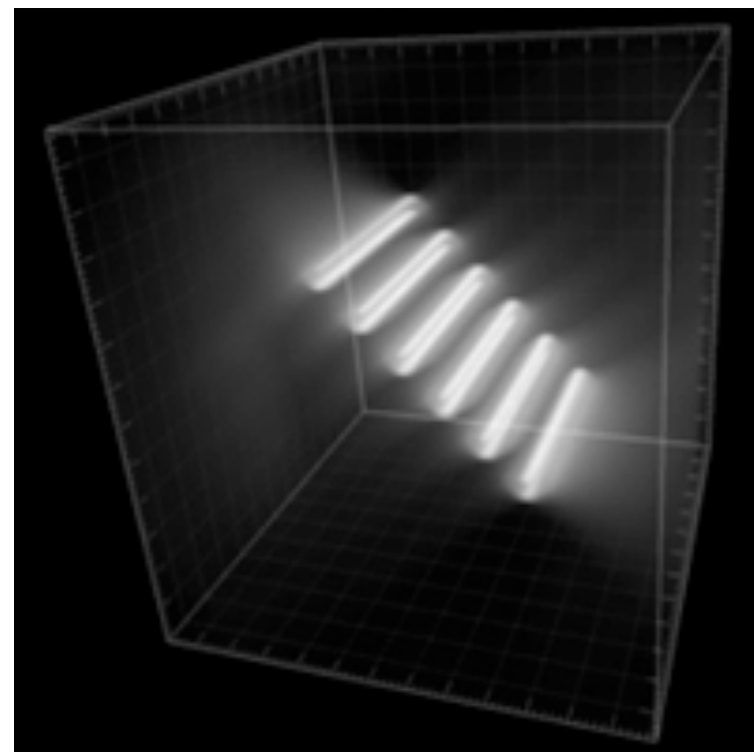
Algorithm-based process
Computational method
Intensive technique



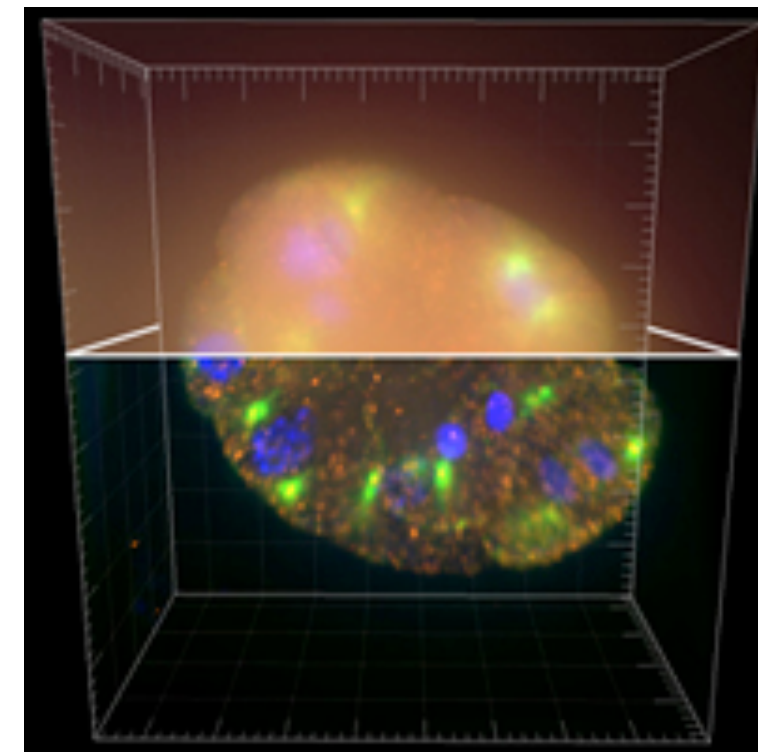
Courtesy of Ferréol Soulez



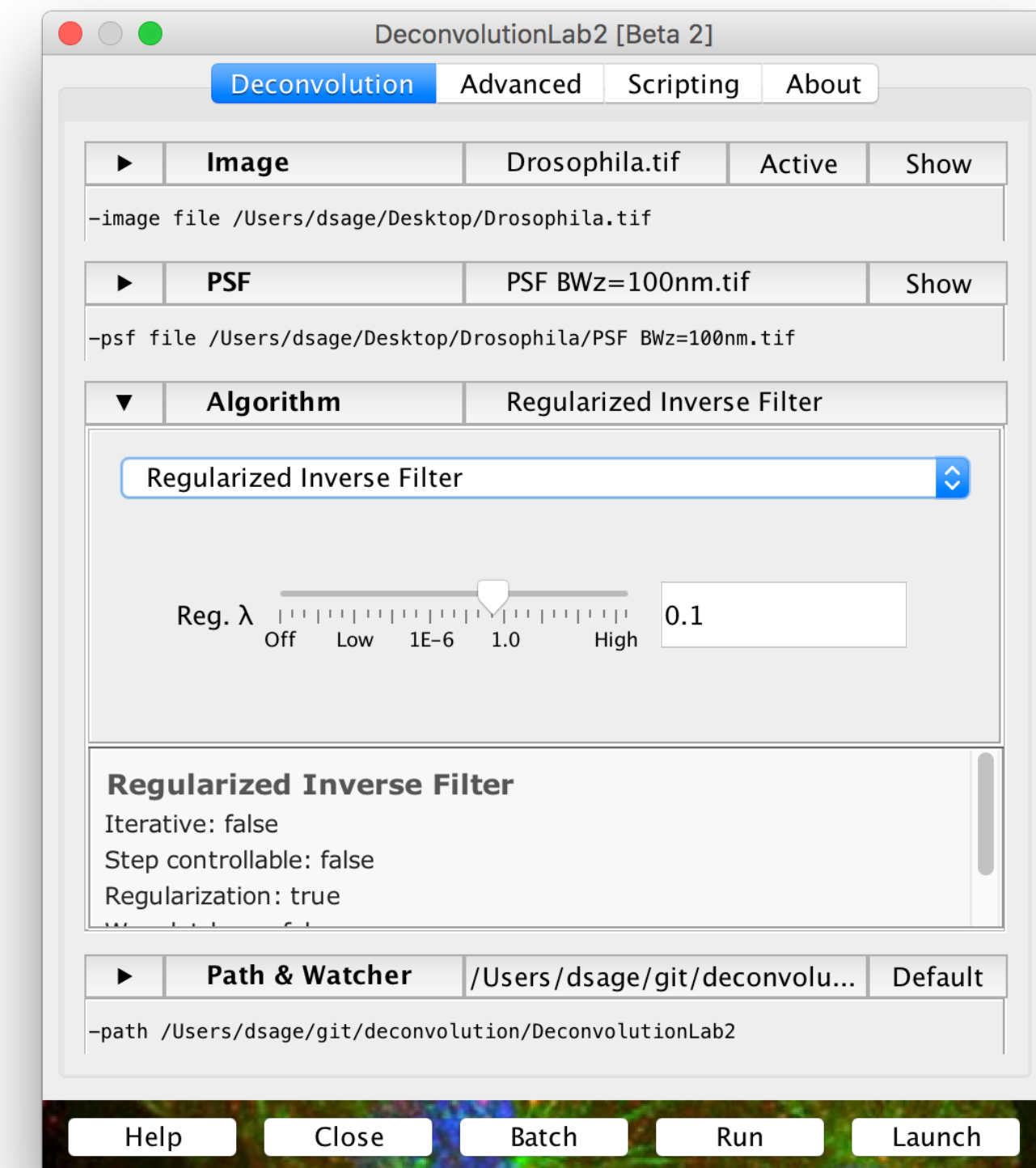
- Purely deconvolution: no pre-, no post-processing
- Pedagogical tool: test the effect of the parameters
- Formula translation close to the theory
- Java platform to host algorithms
- Scripting in Java, MacroJ, Matlab, CLI
- FFT: AcademicFFT (built-in), JTransforms, FFTW
- 3D real datasets, simulation



Synthetic data



Real data



Graphic user interface of
DeconvolutionLab on MacOSX





Easy-to-use

- Choose an image
- Select a PSF
- Choose an algorithm
- Run

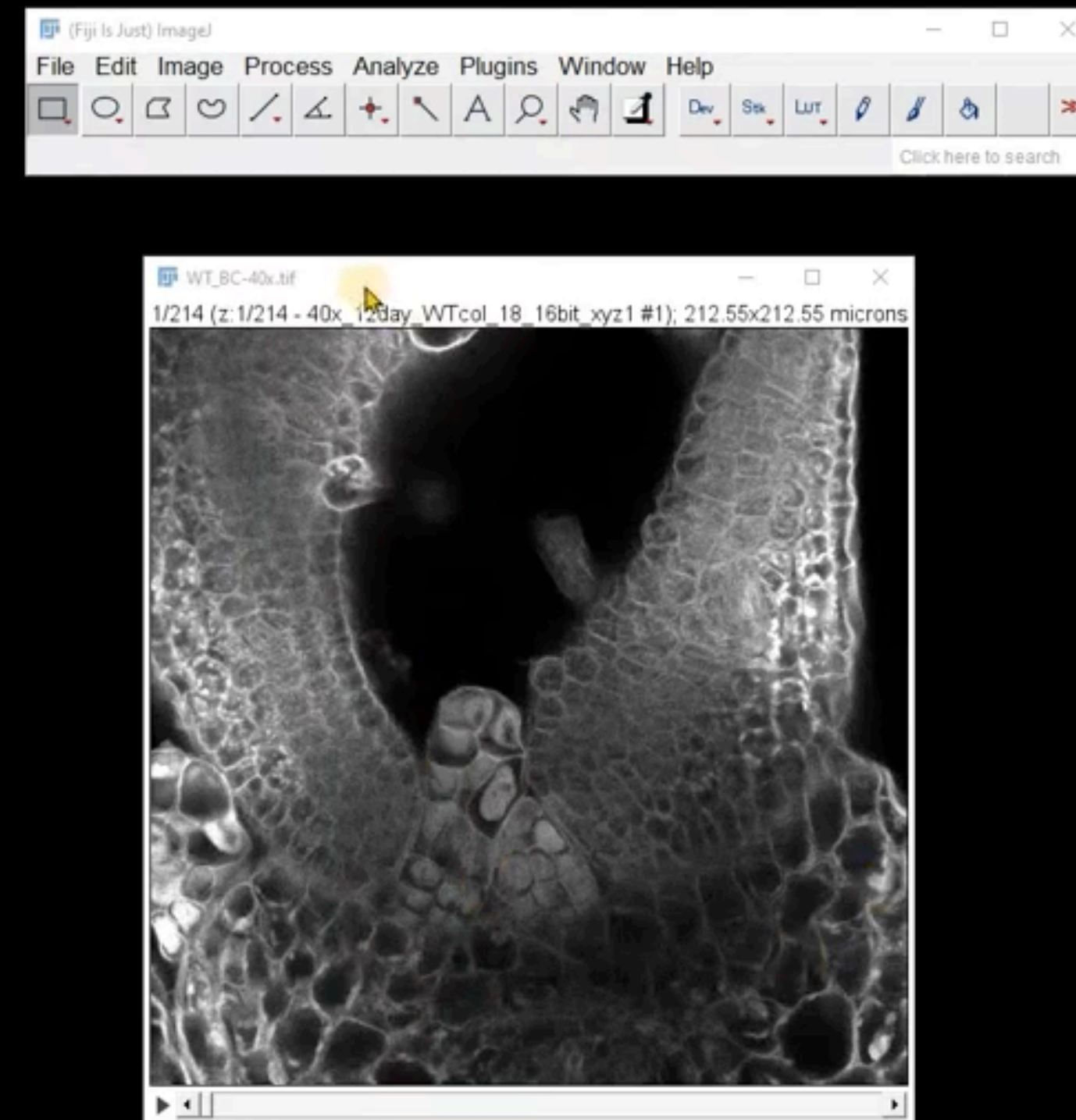


Johanna M. Dela Cruz

@johanna.m.dela-cruz 2.24K
subscribers 86 videos
Listen, watch and learn with me. My
channel is all about optical
microscopy and image
processing & analysis
with FIJI (ImageJ).



This z stack
with 214
optical
sections
was acquired
on a confocal
microscope.

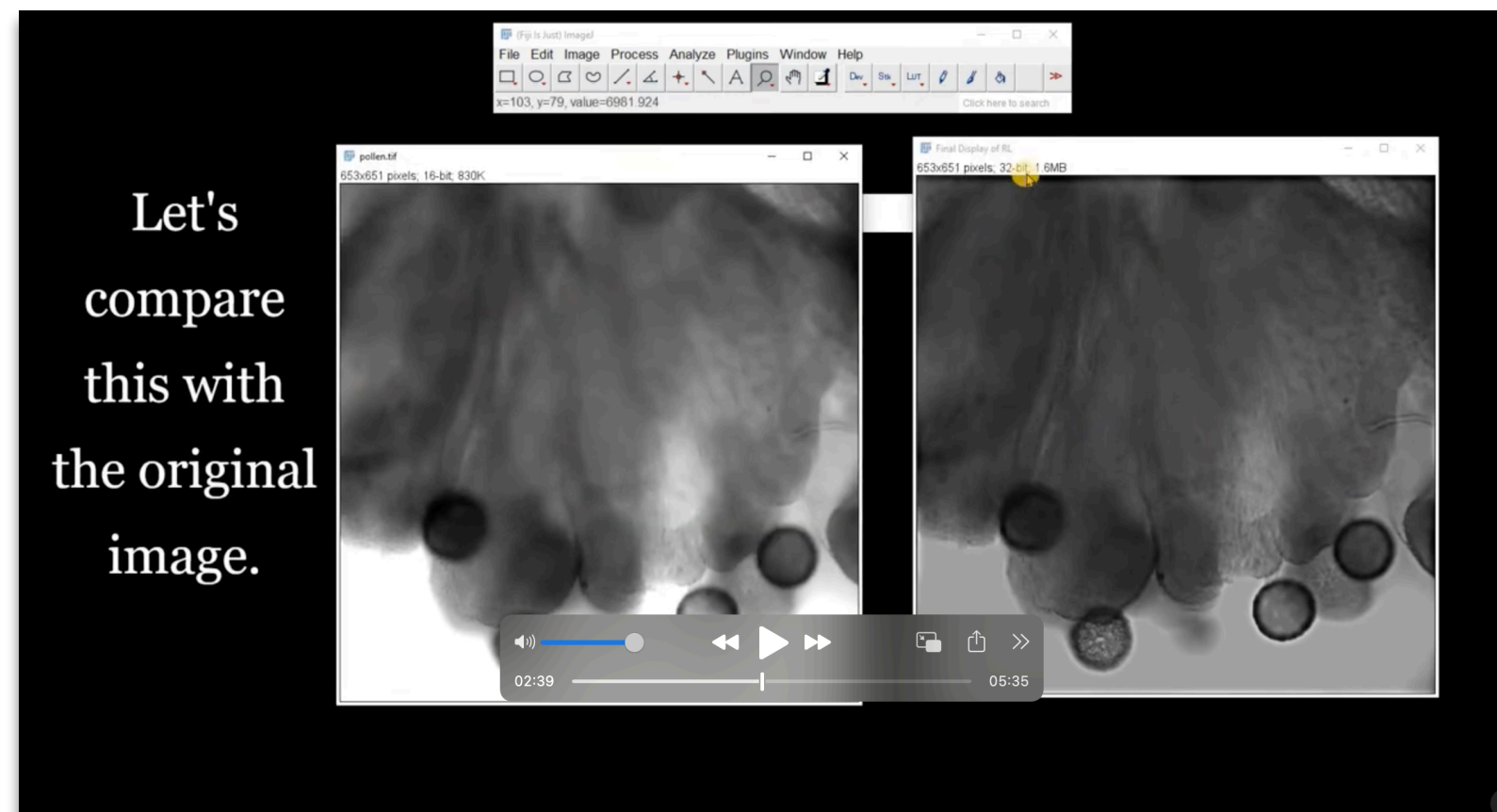


Credits: Johanna M. Dela Cruz

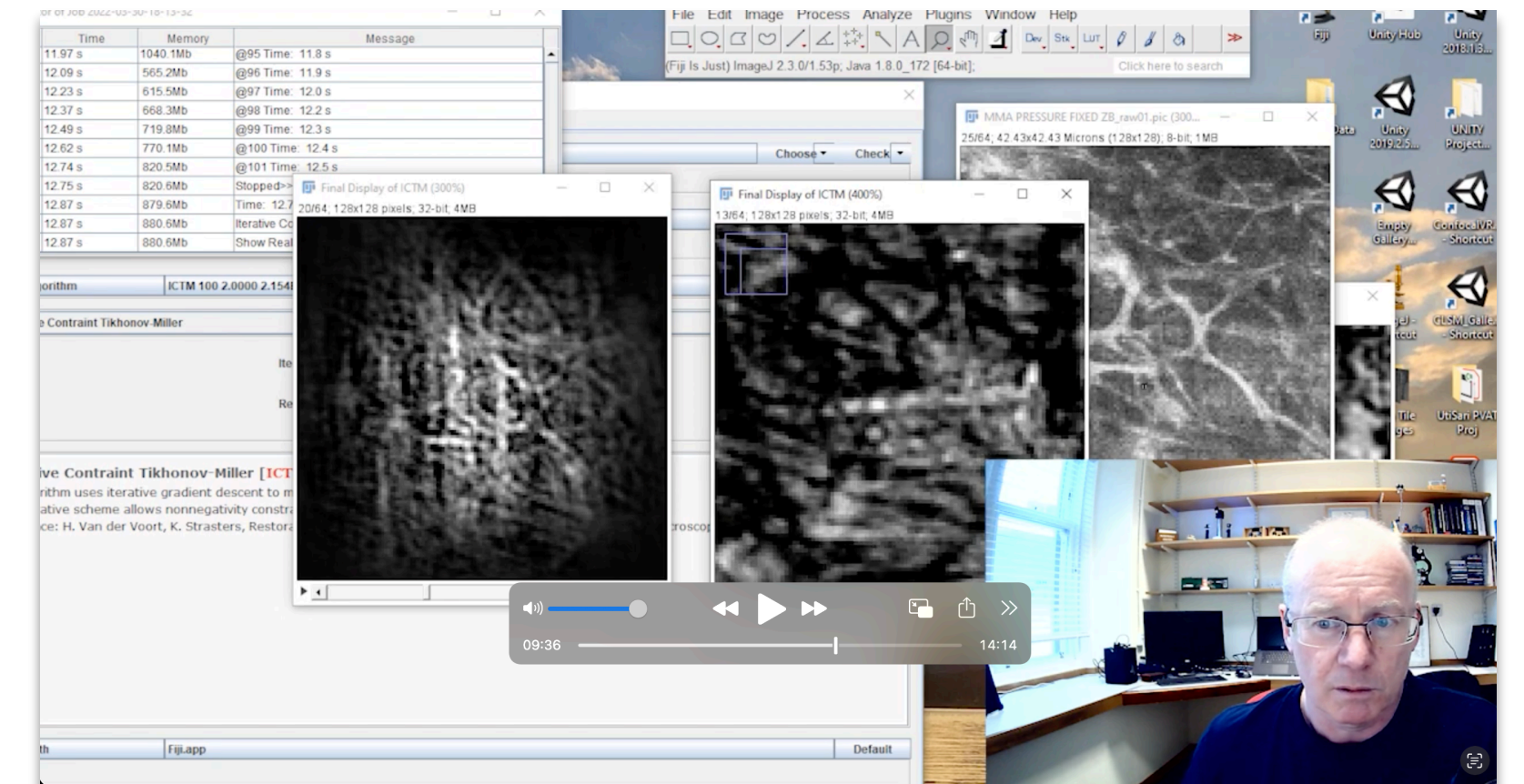


Pedagogic

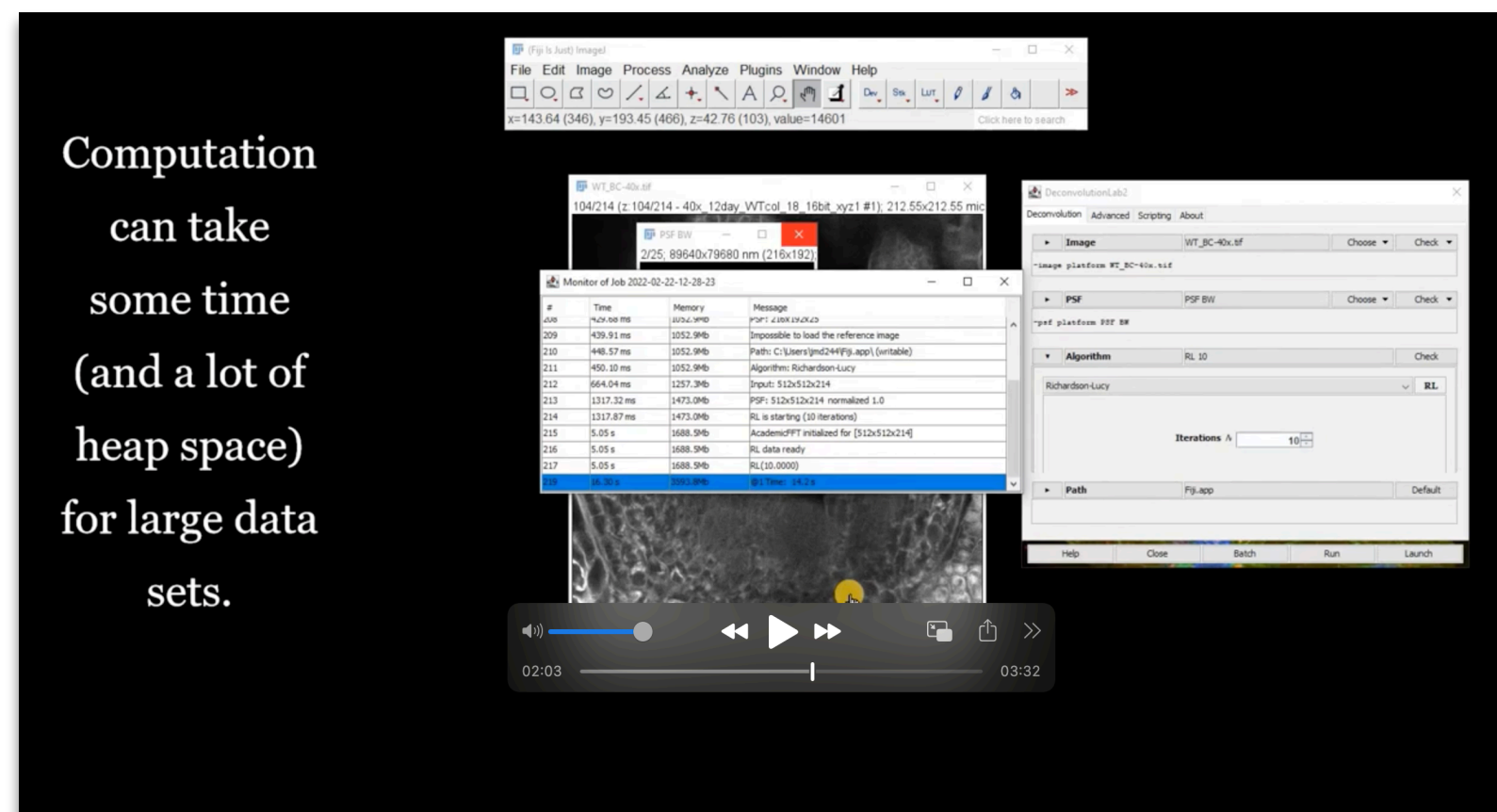
- Video Tutorials
- Run simulations
- GUI
- Test algorithms



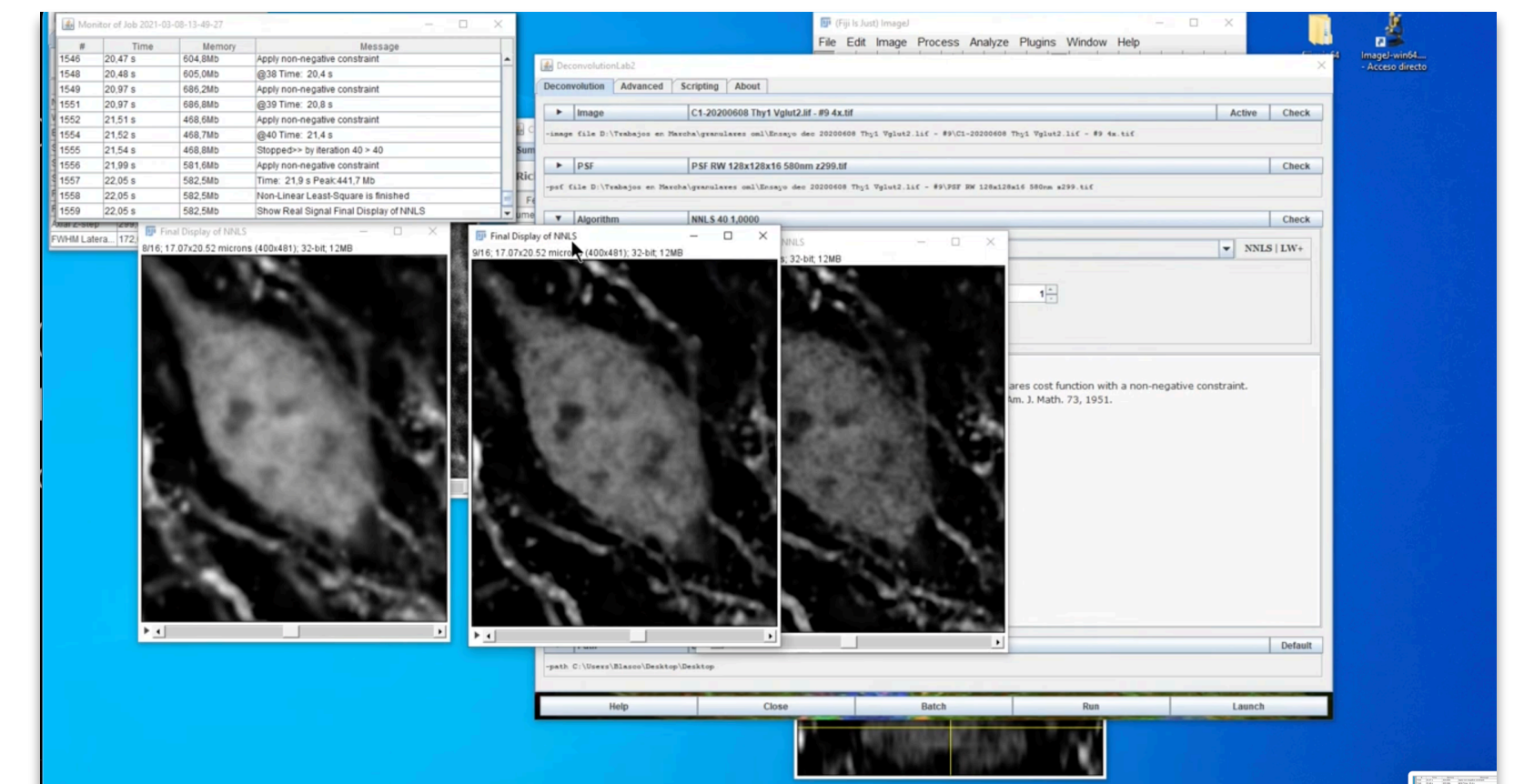
Fiji (ImageJ): Image Deconvolution [Experimental PSF]
Johanna M. Dela Cruz



Fixing blurry images in Fiji/ImageJ (Deconvolution Lab2)
Craig Daly



Fiji (ImageJ): 3D Image Deconvolution [Theoretical PSF]
Johanna M. Dela Cruz



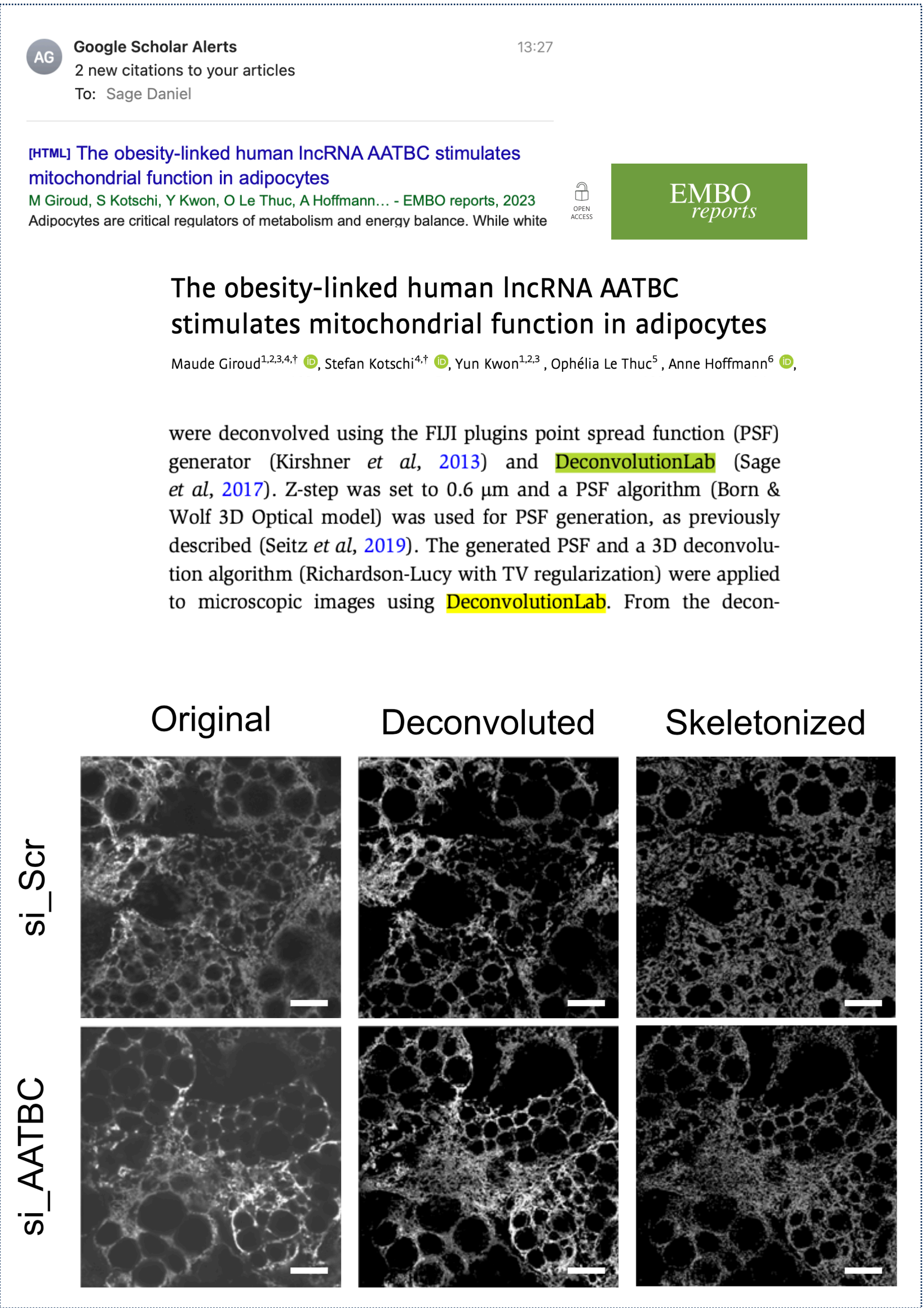
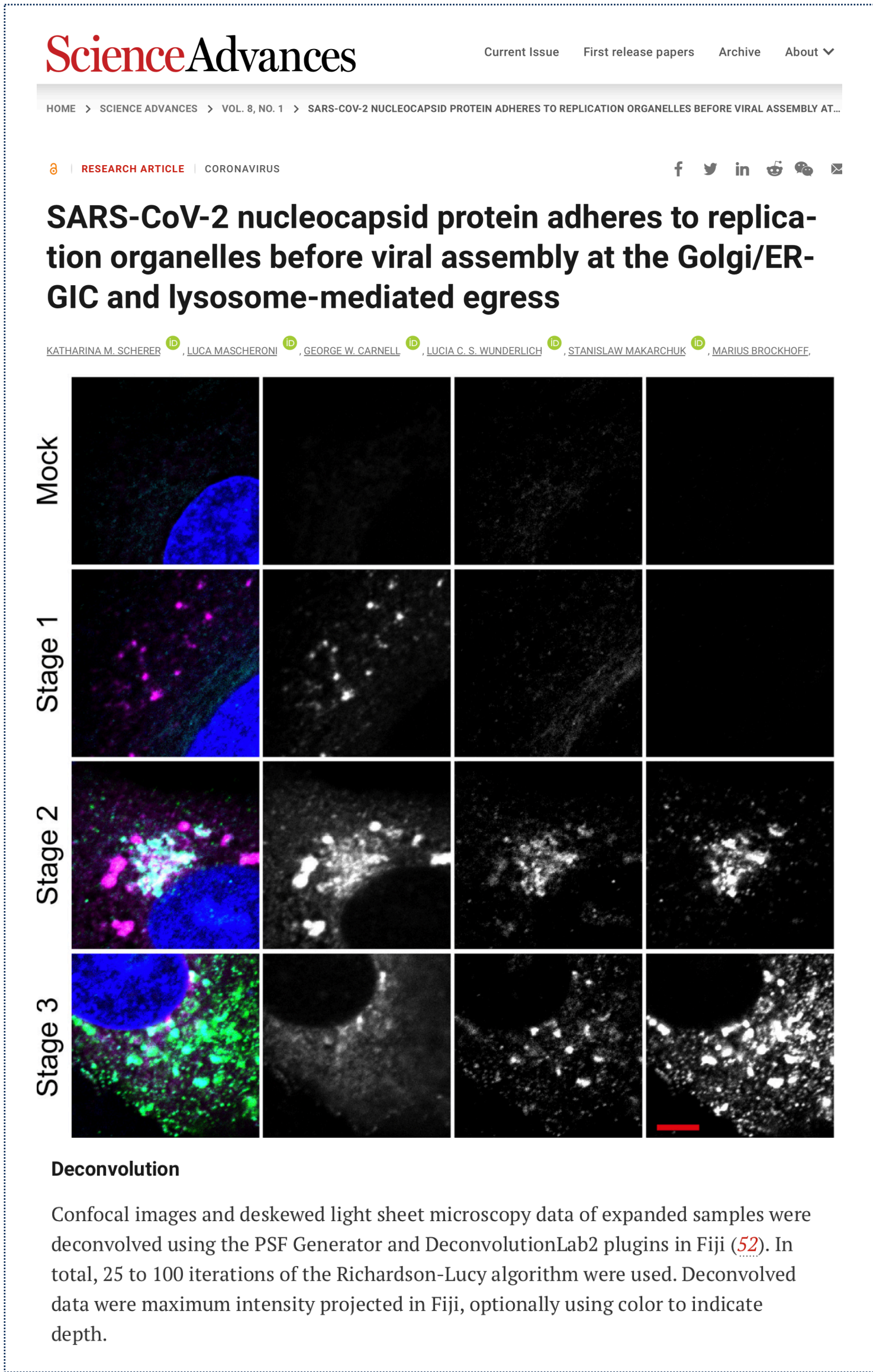
3D Deconvolution Microscopy - YouTube
Jose Miguel Blasco



Real cases

- Scripting
- Integration in pipeline
- Not so slow on CPU

[HTML](#) **DeconvolutionLab2**: An open-source software for deconvolution microscopy
[D Sage](#), [L Donati](#), [F Soulez](#), [D Fortun](#), [G Schmit](#), [A Seitz](#)... - *Methods*, 2017 - Elsevier
... into the framework of **DeconvolutionLab2**. The source code is ... Our goal with **DeconvolutionLab2** is to make deconvolution ... test data on the **DeconvolutionLab2** website. These ...
[SaveCiteCited by 455Related articlesAll 18 versions](#)





$$\tilde{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \left\{ \mathcal{D}(\mathbf{H}\mathbf{x}, \mathbf{y}) + \lambda \mathcal{R}(\mathbf{x}) \right\}$$

M.L.E.
 $\mathbf{1}^T \mathbf{H}\mathbf{x} - \mathbf{y}^T \log(\mathbf{H}\mathbf{x})$

L.S.
 $\|\mathbf{H}\mathbf{x} - \mathbf{y}\|^2$

Non negativity
 $\|\mathbf{H}\mathbf{x} - \mathbf{y}\|^2 + i_{\geq 0}(\mathbf{x})$

hyperparameter
(effect?)

Regularization scheme	Inverser Filter (Direct)	Iterative ↗ Poisson noise	Iterative ↗ Gaussian noise	Constraint "Positivity"
	NIF Naive Inverse Filter	RL Richardson-Lucy	LW Landweber	LW+ Landweber + non-negativity
"Smoothing" $\lambda \ \mathbf{x}\ ^2$ $\lambda \ \mathbf{L}\mathbf{x}\ ^2$	RIF TRIF Regularized Tikhonov Reg.		TM Tikhonov-Miller	ICTM Iterative Constraint Tikhonov-Miller
"Edge preserving" $\lambda \ \mathbf{D}\mathbf{x}\ _1$		RLTV RL with Total Variation		
"Wavelet sparsity" $\lambda \ \mathbf{W}\mathbf{x}\ _1$		ISTA FISTA (Fast) Iterative Shrinkage Thresholding Algorithm		

number of iterations
(stop?)



Inverse Filters

NIF Naive Inverse Filter

$$\hat{f}_{NIF} = \frac{1}{\max(\hat{h}, \epsilon)}$$

Never works in real life

WIF Wiener Inverse Filter

$$\hat{f}_{WIF} = \frac{1}{h(\omega) + \frac{S_n(\omega)}{S_y(\omega)}}$$

WIF Requires noise of signal-to-noise ratio at each frequency

TRIF Tikhonov Regularized Inverse Filter

$$\mathcal{C}(\mathbf{x}) = \|\mathbf{H}\mathbf{x} - \mathbf{y}\|^2 + \lambda \|\mathbf{x}\|^2$$

$$\nabla \mathcal{C}(\mathbf{x}) = 0 \implies 2\mathbf{H}^T(\mathbf{H}\mathbf{x} - \mathbf{y}) + 2\lambda\mathbf{x} = 0$$

$$\mathbf{x} = (\mathbf{H}^T\mathbf{H} + \lambda\mathbf{I})^{-1} \mathbf{H}^T\mathbf{y}$$

$$\hat{f}_{TRIF} = \frac{1}{h(\omega) + \lambda}$$

RIF (Laplacian) Regularized Inverse Filter

$$\mathcal{C}(\mathbf{x}) = \|\mathbf{H}\mathbf{x} - \mathbf{y}\|^2 + \lambda \|\mathbf{L}\mathbf{x}\|^2$$

- Acts as a whitening filter
- Finer controls on most natural images

$$\mathbf{x} = (\mathbf{H}^T\mathbf{H} + \lambda\mathbf{L}^T\mathbf{L})^{-1} \mathbf{H}^T\mathbf{y}$$

$$\hat{f}_{LRIF} = \hat{f}_{RIF} = \frac{1}{h(\omega) + \lambda \omega^2}$$



Regularized Inverse Filter

TRIF Tikhonov Regularized Inverse Filter

$$\mathcal{C}(\mathbf{x}) = \|\mathbf{y} - \mathbf{H}\mathbf{x}\|^2 + \lambda\|\mathbf{x}\|_2^2$$

$$\mathbf{x} = (\mathbf{H}^T \mathbf{H} + \lambda \mathbf{I})^{-1} \mathbf{H}^T \mathbf{y}$$

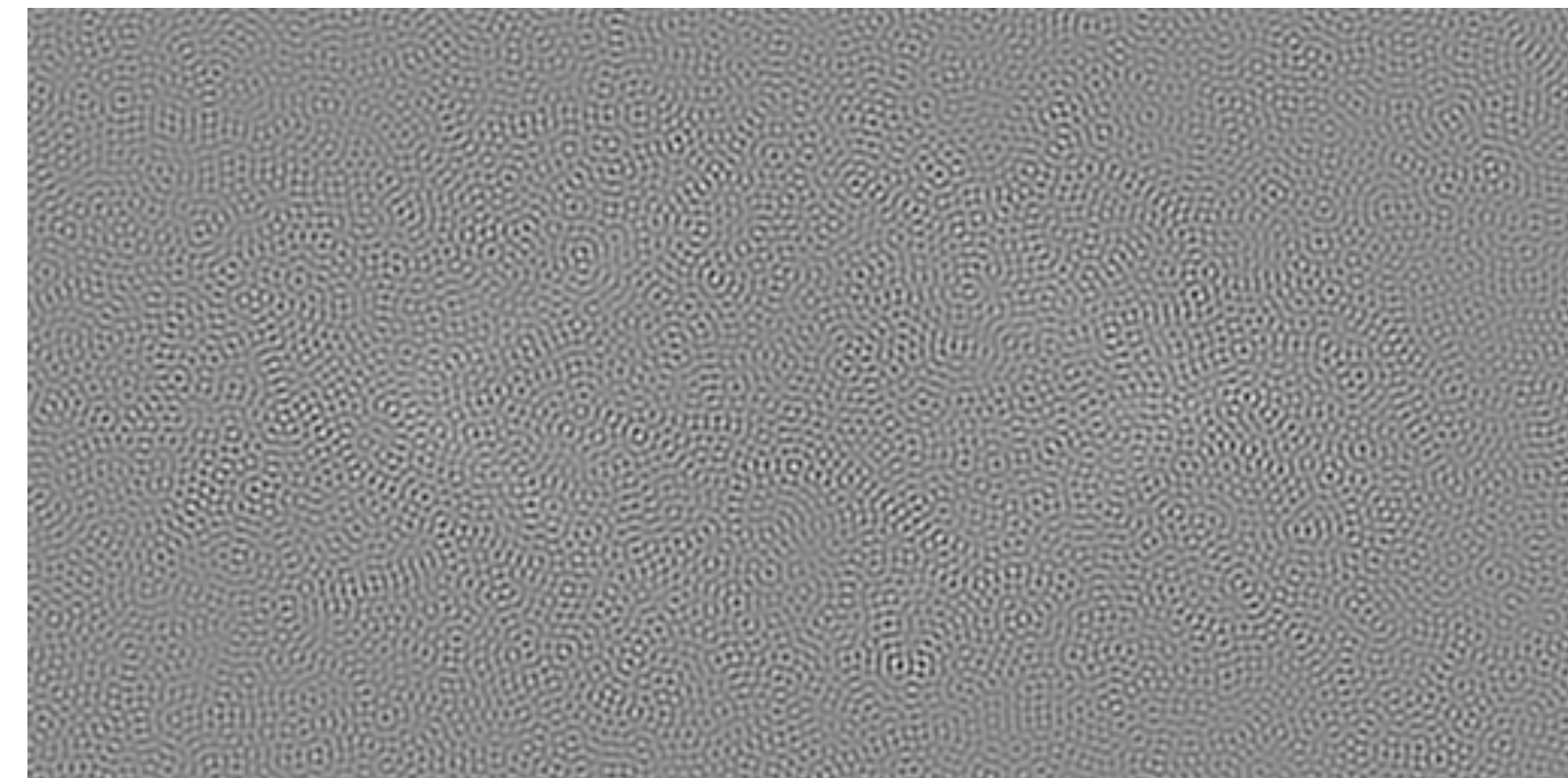
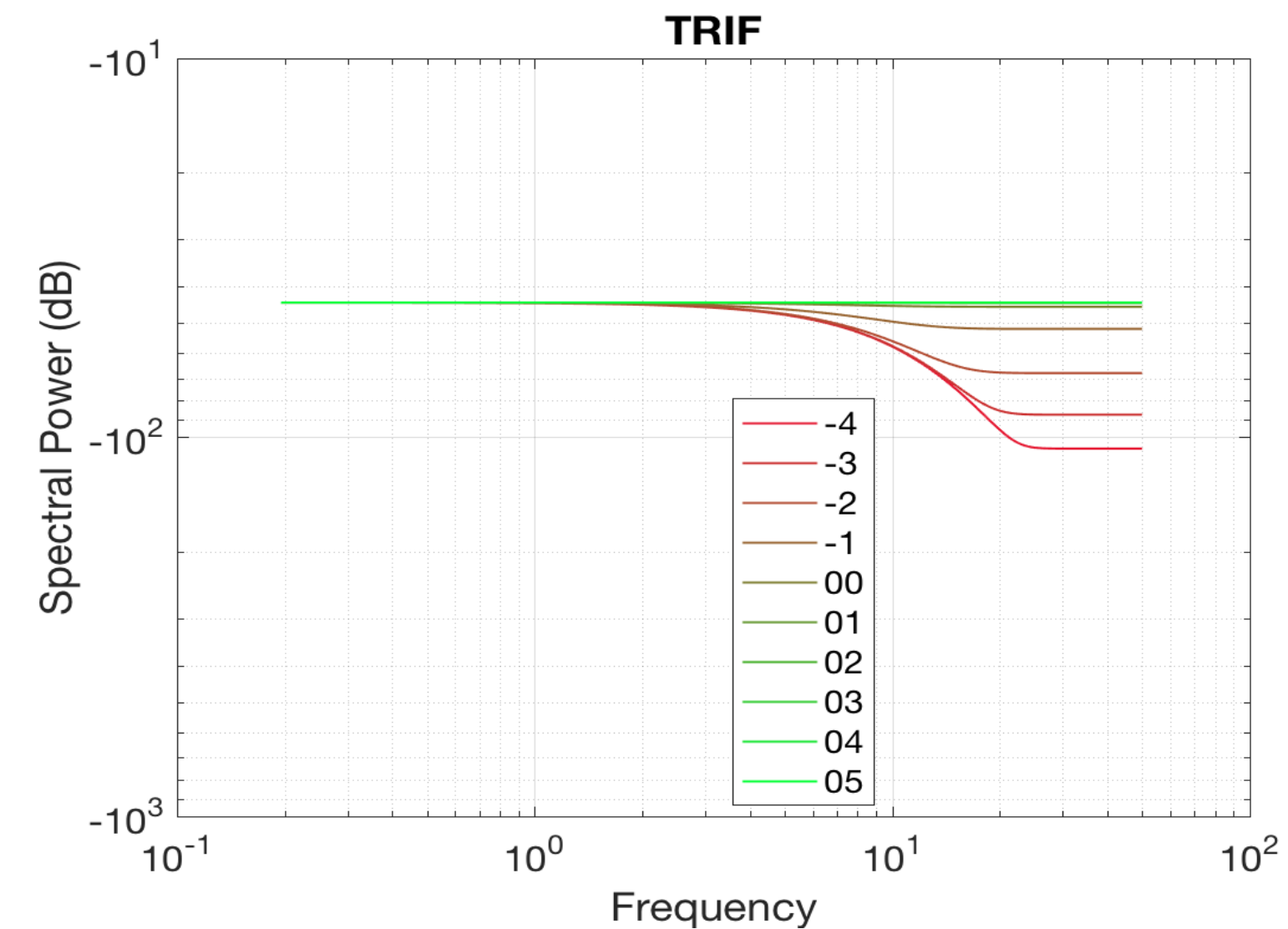
- Compensate the high-pass filter effect
- L2 norm

RIF (Laplacian) Regularized Inverse Filter

$$\mathcal{C}(\mathbf{x}) = \|\mathbf{y} - \mathbf{H}\mathbf{x}\|^2 + \lambda\|\mathbf{L}\mathbf{x}\|_2^2$$

$$\mathbf{x} = (\mathbf{H}^T \mathbf{H} - \lambda \mathbf{L}^T \mathbf{L})^{-1} \mathbf{H}^T \mathbf{y}$$

- Act as a whitening filter
- Finer control on most natural images





Steepest Gradient Descent

LW Landweber iteration

[Landweber, 1951]

$$\mathcal{C}(\mathbf{x}) = \|\mathbf{y} - \mathbf{H}\mathbf{x}\|^2$$

$$\mathbf{x}^{k+1} = (\mathbf{I} - \gamma \mathbf{H}^T \mathbf{H}) \mathbf{x}^k + \gamma \mathbf{H}^T \mathbf{y}$$

- LWID, Landweber iterative deconv.
- Least-square minimization
- Controllable steepest ($\gamma < 2$)
- Dominant Gaussian noise

TM Tikhonov-Miller

$$\mathcal{C}(\mathbf{x}) = \|\mathbf{y} - \mathbf{H}\mathbf{x}\|^2 + \lambda \|\mathbf{L}\mathbf{x}\|_2^2$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \gamma \left(\mathbf{H}^T \mathbf{y} - (\mathbf{H}^T \mathbf{H} + \lambda \mathbf{L}^T \mathbf{L}) \mathbf{x}^{(k)} \right)$$

- Tikhonov regularization

Landweber + positivity

LW+

Projected solution

$$\mathbf{x}^{k+1} = \mathcal{P} \left\{ (\mathbf{I} - \gamma \mathbf{H}^T \mathbf{H}) \mathbf{x}^k + \gamma \mathbf{H}^T \mathbf{y} \right\}$$

- Known also NNLS
- Non-negative constraint $\Rightarrow$ slow down!

Iterative Constrained Tikhonov-Miller

ICTM

[Kempen, 1996]

Projected solution

$$\mathbf{x}^{(k+1)} = \mathcal{P} \left\{ \mathbf{x}^{(k)} + \gamma \left(\mathbf{H}^T \mathbf{y} - (\mathbf{H}^T \mathbf{H} + \lambda \mathbf{L}^T \mathbf{L}) \mathbf{x}^{(k)} \right) \right\}$$



Richardson-Lucy

RL Richardson-Lucy

[Richarsdon, 1972, Lucy 1974]

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} \times \mathbf{H}^T \left(\frac{\mathbf{y}}{\mathbf{H}\mathbf{x}^{(k)}} \right)$$

- Statistically interpretation
- Poisson noise
- Assumption of positive signals
- Maximum likelihood estimator (MLE)
- Slow, iteration in the spatial domain
- One parameter to tune (number of iterations)

RLTV Richardson-Lucy with Total Variation

[Dey, 2006]

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} \times \mathbf{H}^T \left(\frac{\mathbf{y}}{\mathbf{H}\mathbf{x}^{(k)}} \right) + \lambda \|\mathbf{D}\mathbf{x}\|_1$$

- Preserve the edges
- How to balance the TV and the deconvolution



Promote Sparsity

ISTA Iterative Soft Threshold Algorithm

FISTA Fast Iterative Soft Threshold Algorithm

[Beck 2009]

$$\mathcal{C}(\mathbf{x}) = \|\mathbf{y} - \mathbf{H}\mathbf{x}\|^2 + \lambda\|\mathbf{W}\mathbf{x}\|_1$$

- Fast convergence
- Preserve the edges
- Preserve discontinuities

$$\mathbf{z}^{(k+1)} = \mathbf{s}^{(k)} - \gamma\mathbf{H}^T(\mathbf{H}\mathbf{s}^{(k)} - \mathbf{y})$$

$$\mathbf{x}^{(k+1)} = \mathbf{W}^T\mathcal{T}(\mathbf{W}\mathbf{z}^{(k+1)}, \gamma\lambda)$$

$$p^{(k+1)} = \frac{1}{2} \left(1 + \sqrt{1 + 4p^{(k)2}} \right)$$

$$\mathbf{s}^{(k+1)} = \mathbf{x}^{(k+1)} + \frac{p^{(k)} - 1}{p^{(k+1)}}(\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)})$$

- Soft-threshold in the wavelet domain
- Haar wavelets, Spline wavelets 1, 3, 5



RIF (Laplacian) Regularized Inverse Filter

$$\mathcal{C}(\mathbf{x}) = \|\mathbf{H}\mathbf{x} - \mathbf{y}\|^2 + \lambda \|\mathbf{L}\mathbf{x}\|^2$$

$$\hat{f}_{LRIF} = \hat{f}_{RIF} = \frac{1}{h(\omega) + \lambda \omega^2}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \gamma (\mathbf{H}^T \mathbf{y} - (\mathbf{H}^T \mathbf{H} + \lambda \mathbf{L}^T \mathbf{L}) \mathbf{x}^{(k)})$$

$$\mathbf{A} = \gamma (\mathbf{H}^T \mathbf{H} + \lambda \mathbf{L}^T \mathbf{L})$$

$$\mathbf{G} = \gamma \mathbf{H}^T \mathbf{y}$$

$$\mathbf{x}^{(k+1)} = \mathbf{A}\mathbf{x}^{(k)} + \mathbf{G} \quad \text{.....} \rightarrow$$

```
public RealSignal call() throws Exception {
    ComplexSignal Y = fft.transform(y);
    ComplexSignal H = fft.transform(h);
    ComplexSignal A = Operations.delta(gamma, H);
    ComplexSignal L = SignalFactory.fourierLaplacian(Y.nx, Y.ny, Y.nz);
    ComplexSignal L2 = Operations.multiplyConj(lambda*gamma, L, L);
    A.minus(L2);
    ComplexSignal G = Operations.multiplyConj(gamma, H, Y);
    ComplexSignal X = G.duplicate();
    while(!controller.ends(X)) {
        X.times(A);
        X.plus(G);
    }
    RealSignal x = fft.inverse(X);
    return x;
}
```